

Prompt Engineering with ChatGPT

Prompt Best Practices

Providing Context using Delimiters

- Use delimiters to clearly separate context, instruction, and constraints in prompts. For example:
 - Context: [brief context]
 - Task: [specific ask]
 - Constraints: [format, length, style, safety rules]
- When requesting or delivering content, break tasks into smaller chunks (task splitting) to improve accuracy and traceability.
- Build prompts with a clear persona when appropriate, e.g., “You are a concise technical writer” or “You are a data scientist explaining to a non-technical audience.”
- For multimodal or complex outputs, outline expectations for any graphs, charts, or images and specify data formats (JSON, CSV, markdown tables, etc.).
- Prioritize security: avoid exposing secrets, use redaction when sharing samples, and remind about data privacy when handling sensitive inputs.

Advanced Techniques

Persona

- A persona is a clear, pretend character or role you want the AI to act as.
- It’s like telling the AI “You are a concise technical writer” or “You are a friendly tutor.”
- The idea is to shape how the AI responds: tone, style, level of detail, and what information to emphasize.
- Using a persona helps keep answers consistent and fits the audience or task.

If you want, I can give you quick examples:

- Persona: Technical writer for developers
 - Style: precise, brief, with code examples
- Persona: Friendly tutor for beginners
 - Style: patient, simple analogies, lots of step-by-step guidance
- Persona: Data scientist explaining to non-technical audience
 - Style: plain language, visuals-ready explanations, minimal jargon

e.g.

- Role: You are Ronnie Sheer’s prompt-engineering assistant, specializing in crafting precise, actionable prompts for AI systems.
- Core strengths:

- Contextual prompting and delimiter-driven prompts
- Task decomposition (breaking complex requests into tractable chunks)
- Persona-driven communication (tailoring tone and audience)
- Multimodal awareness (clear expectations for text, images, and data formats)
- Security-first mindset (redaction and privacy by default)
- Tone and style:
 - Clear, concise, and structured
 - Audience-aware: adapts complexity to the user's expertise
 - Uses delimiters to separate Context, Task, and Constraints
- Delimiter guidelines:
 - Context: [Brief context here]
 - Task: [Specific ask here]
 - Constraints: [Format requirements, length, style, safety rules]
- Prompting rules:
 - Break complex requests into 3–5 sub-tasks when possible
 - Propose persona variants when helpful (e.g., “You are a concise technical writer”)
 - Specify data outputs in preferred formats (JSON, CSV, Markdown tables, etc.)
 - Prioritize security: redact secrets, avoid sharing sensitive data
- Examples of persona prompts:
 - Summary task:
 - Context: [Provide a brief summary of the document or content]
 - Task: “Summarize the key points at a high level, then provide 3 actionable takeaways.”
 - Constraints: “Limit to 5 bullet points; use plain language for a non-technical audience.”
 - Code explanation task:
 - Context: [Code snippet or repository context]
 - Task: “Explain what the code does, step by step, and annotate potential edge cases.”
 - Constraints: “Include Python-style pseudocode for critical sections; avoid external dependencies’ specifics.”
 - Design critique task:
 - Context: [Design brief or UI mock]

- Task: “Provide a constructive critique focused on usability, accessibility, and performance.”
- Constraints: “Suggest 2–3 concrete changes with rationale.”

How to apply this persona

- When you need to craft prompts for a user story, start with:
 - Context: [User-facing context]
 - Task: [Clear objective]
 - Constraints: [Format, safety, tone]
- For multi-part requests, present a 3-step plan:
 1. Define scope and success criteria
 2. Split into sub-tasks
 3. Deliver outputs in specified formats with quick checklists

Splitting up Complex tasks

- Break down complex, multi-step requests into smaller, manageable sub-tasks to improve clarity, accuracy, and tracking.
- Why splitting helps:
 - Reduces cognitive load for both the AI and the user.
 - Makes it easier to verify progress and correctness at each step.
 - Improves error handling by isolating issues to individual sub-tasks.
- How to split effectively:
 - Define scope and success criteria at the outset.
 - Decompose into 3–5 sub-tasks whenever possible (the video emphasizes a practical upper bound to maintain focus).
 - Sequence sub-tasks so dependencies flow logically (some tasks can run in parallel when independent).
- Deliberate planning steps demonstrated:
 - Create a 3-step plan before execution:
 1. Clarify objectives and success metrics.
 2. Break the task into sub-tasks with clear deliverables.
 3. Execute sub-tasks and synthesize results, with a final check for coherence.
- Concrete examples shown:
 - A complex prompt is re-framed into: Context, Task, and Constraints, each with explicit details.

- A multi-part user request (e.g., designing a study guide) is divided into research, outline creation, and draft refinement.
- Best practices highlighted:
 - Use personas to tailor expectations and language for each sub-task.
 - Maintain a running checklist or rubric to evaluate each deliverable.
 - Employ redaction and privacy considerations when handling sensitive inputs.
- Outcomes you should aim for:
 - Clear sub-task outputs that can be recombined into the final result.
 - Traceable reasoning: you can audit how each sub-task contributed to the overall goal.
 - A final deliverable that integrates all sub-task results consistently.

Custom instructions

- Custom instructions tailor an AI assistant's behavior to specific guidelines, audiences, or tasks, ensuring outputs align with user needs and organizational standards.
- Why use custom instructions:
 - Personalization: adjusts tone, level of detail, and formatting to match the user or audience.
 - Consistency: enforces a predictable style across outputs.
 - Safety and governance: embeds policy considerations (privacy, sensitive data handling, redaction).
- Common components shown:
 - Role definition: specifying the assistant's purpose and persona.
 - Audience and tone: selecting target audience (expert, non-technical, executive) and the desired voice.
 - Output preferences: preferred formats (JSON, Markdown, tables), level of granularity, and examples of expected structure.
 - Constraints and guardrails: rules to avoid sharing secrets, to redact sensitive data, and to adhere to security guidelines.
- How to implement custom instructions:
 - Start with a clear, concise instruction block that defines role and expectations.
 - Pair with a task-specific instruction for the current prompt to guide execution.
 - Include formatting templates or delimiters to ensure consistent outputs (e.g., Context, Task, Constraints blocks).
- Examples illustrated:
 - Example 1: "You are a concise technical writer for a non-technical audience; format responses in bullet lists with plain language."

- Example 2: “You are a data scientist explaining to executives; provide high-level insights first, then a brief appendix with key metrics.”
- Example 3: “Security-first mode: redact PII, avoid sharing credentials, and flag any risky prompts.”

Potential benefits highlighted:

- Faster alignment on expectations for new tasks.
- Improved auditability by clearly separating role, instructions, and constraints.
- Better handling of multimodal outputs by specifying data formats and visualization needs.

Leveraging multi-modality

- Multi-modality refers to the AI system’s ability to understand and generate across multiple data types (text, images, audio, etc.) and to coordinate responses that integrate these formats seamlessly.
- Why multimodality matters:
 - Enables richer, more actionable outputs by combining information from different channels (e.g., text explanations paired with visuals or diagrams).
 - Improves problem-solving for complex tasks that benefit from visual or data-driven context.
 - Supports accessibility and clearer communication by offering multiple representations of the same insight.
- Key capabilities likely highlighted:
 - Text-to-Image: Generating or interpreting images to accompany explanations or prompts (e.g., visuals that illustrate a concept).
 - Image Understanding: Analyzing uploaded images to extract relevant data, annotations, or insights to augment text responses.
 - Coordinated outputs: Providing synchronized text and visuals, charts, or diagrams as part of a single deliverable.
- How to leverage in prompts:
 - Clearly specify expected modalities in the output (e.g., “Provide a concise explanation in text plus a simple diagram in Markdown”).
 - Use prompts that outline data formats for each modality (e.g., JSON for data, Markdown for narrative, a simple SVG for diagrams).
 - Define success criteria that include the accuracy and usefulness of each modality (e.g., correctness of described visuals, alignment between text and images).
- Practical examples shown:
 - Prompt that requests both a textual summary and a complementary diagram or chart to illustrate key points.

- Scenarios where a user uploads an image or data file and the AI annotates or augments the textual analysis with visuals or data visualizations.
- Best practices highlighted:
 - Explicit modality contracts: tell the model what to output in each channel.
 - Consistency across modalities: ensure the narrative and visuals tell a coherent story.
 - Accessibility considerations: provide alt-text for images and simple, clear description.

Socratic prompting

- Socratic prompting uses a guided, question-led approach to elicit deeper thinking, reveal assumptions, and steer the user toward clearer reasoning and better problem framing.
- Why it matters:
 - Improves critical thinking and self-reflection in both the user and the AI.
 - Helps uncover hidden assumptions and edge cases before producing a final answer.
 - Fosters iterative refinement without giving away the solution too early.
- Core techniques likely highlighted:
 - Asking progressive questions: start with broad queries and drill down with targeted follow-ups.
 - Clarifying assumptions: prompt the user to surface what they're taking for granted.
 - Challenging implications: explore consequences, trade-offs, and alternative viewpoints.
 - Redirecting when needed: steer back to the objective if the discussion veers off course.
- How to implement Socratic prompts:
 - Begin with a high-level question to frame the goal.
 - Use a sequence of prompts that require justification, evidence, and alternatives.
 - Incorporate reflection prompts (e.g., "What would change if X were true?") to test robustness.
- Practical prompts structure:
 - Context: [Brief setup or goal]
 - Task: [Series of questions designed to elicit reasoning and justification]
 - Constraints: [Required formats, depth of explanation, and safety guardrails]
- Examples illustrated:
 - Educational scenario: guiding a student to derive a solution step-by-step rather than giving the answer.
 - Decision support: probing assumptions behind a proposed course of action and weighing alternatives.
 - Policy or design critique: prompting for evidence, counterarguments, and risk assessment.
- Best practices highlighted:
 - Maintain a calm, exploratory tone to encourage honest reasoning.
 - Use a mix of open-ended and targeted questions to balance depth and direction.
 - Capture and structure the dialogue so it's auditable and reproducible.
 - Combine with a final synthesis step that distills the reasoning into a clear conclusion.

Dall-E 3 in Chatgpt

What is Dall-E ?

- DALL-E is a multimodal AI model that generates images from natural language prompts, enabling creative visualizations from textual descriptions.
- Key capabilities highlighted:
 - Text-to-image generation: turning descriptive prompts into novel images.
 - Image editing: modifying existing images based on textual instructions (in some versions, like in-painting or out-painting).
 - Conceptual alignment: translating abstract ideas (styles, objects, scenes) into coherent visuals.
- How it fits into a broader AI toolkit:
 - Complements text-based outputs by producing visuals that illustrate concepts, designs, or data.
 - Supports ideation, mockups, storytelling, and visual communication in interdisciplinary workflows.
- Practical workflow concepts likely demonstrated:
 - Crafting precise prompts to guide style, composition, color palette, and level of detail.
 - Iterative refinement: generating multiple variants and selecting or blending elements.
 - Managing constraints like copyright, originality, and ethical use of generated imagery.
- Best practices emphasized:
 - Use clear, descriptive prompts with concrete attributes (subject, setting, style, mood).
 - Include desired output characteristics (aspect ratio, resolution, color scheme) to reduce back-and-forth.
 - Consider accessibility and usability: provide alt-text for generated images and ensure outputs are interpretable.
- Example prompt structure:
 - Context: “Describe a futuristic product concept.”
 - Task: “Generate a high-resolution image showing the product in a desktop setting, in a clean, minimalist style.”
 - Constraints: “SVG or PNG output; 2048x1024 resolution; include alt-text describing key features.”
- Expected outcomes:
 - A set of image variants aligned with the prompt.
 - Clear rationale for why a chosen variant best fits the brief, aiding decision-making and iteration.

Dall-E 3 Best Practices

- DALL-E 3 best practices focus on maximizing image quality, relevance, and ethical use when generating visuals from natural language prompts. Emphasis is on precision, iteration, and alignment with intended audience and constraints.
- Key capabilities and focus areas likely highlighted:
 - Prompt precision: crafting clear, descriptive prompts to steer style, composition, lighting, and details for consistent results.
 - Iterative refinement: generating multiple variants, selecting top candidates, and blending elements to converge on the desired output.

- Safety and ethical considerations: avoiding misinformation, respecting copyright, and applying redaction or alterations when necessary to reduce misuse.
- Prompt construction guidelines typical for DALL-E 3:
 - Start with a concrete subject and setting, specify style and mood, and include constraints like aspect ratio and resolution.
 - Use explicit attributes for composition (foreground/background, camera angle, lighting) to reduce ambiguity.
 - Include success criteria (e.g., alignment with brand guidelines, legibility of details in thumbnails).
- Multimodal alignment concepts:
 - If combining text and visuals, ensure the image output complements any accompanying text and data.
 - When providing output in a workflow, specify the format for the image (SVG/PNG/JPG) and any metadata or alt text for accessibility.
- Practical workflow examples:
 - A structured prompt sequence that yields a few variant images, with a simple rubric for choosing the best one.
 - Using prompts that enforce color palettes, typography cues, or design system tokens to maintain consistency with a given brand.
- Best practices highlighted:
 - Explicitly describe what to include and exclude (e.g., “no watermark, high contrast, high detail in product texture”).
 - Plan for post-processing: specify if vectorization, editing, or compositing in a later step is expected.
 - Accessibility considerations: provide alt text and ensure visual clarity for diverse audiences.
- Outcomes you should aim for:
 - A set of high-quality image variants tightly aligned with the brief.
 - Clear rationale for why a chosen variant best fits the brief, aiding decision-making and iteration.

Security Considerations

ChatGPT Security Considerations

- Identify practical security considerations when using ChatGPT and similar AI systems, including data handling, privacy, and risk mitigation.
- Key security themes likely addressed:
 - Data privacy and leakage: how prompts, inputs, and model outputs can expose sensitive information; best practices to minimize exposure.

- Redaction and sanitization: strategies to obfuscate or remove PII or secrets before feeding data into the model.
- Access controls and authentication: ensuring that prompt-based tools are used by authorized users and within trusted environments.
- Prompt safety and content governance: designing prompts to avoid instructing the model to reveal or generate sensitive information.
- Auditability and traceability: maintaining logs and having an auditable trail of prompts, outputs, and decisions for compliance.
- Practical guidelines likely highlighted:
 - Use redaction techniques for any sensitive data in prompts.
 - Define clear data-handling policies for what can be shared with AI systems.
 - Employ minimum-privilege usage: limit model access to only the data necessary for the task.
 - Separate environments for development and production prompts to reduce risk.
- Workflow implications:
 - Build prompts with security in mind from the start (security-by-design).
 - Include checks or gates before final outputs are shared externally.
 - Regular review of prompts and data flows to identify potential leakage vectors.
- Examples typically shown:
 - A prompt design exercise that demonstrates redacting sensitive fields while preserving enough context for the task.
 - A checklist for securely handling prompts and outputs during a team collaboration scenario.
- Best practices summarized:
 - Treat prompts and responses as potentially sensitive data; plan redaction and data minimization.
 - Document data flows and security controls for AI-assisted workflows.
 - Align with organizational governance, compliance requirements, and data privacy laws.
- Outcomes you should aim for:
 - Safer, compliant use of ChatGPT in real workflows.
 - Clear guidance on what can be shared with or asked of the model.
 - A reusable security checklist and prompts that embed privacy safeguards.

Plans and Data Controls

- This video discusses how to plan AI tasks and implement data controls to ensure secure, reliable, and auditable workflows when using AI systems. It emphasizes structuring prompts, defining data boundaries, and establishing governance around inputs, outputs, and processes.
- Planning and task framing:
 - Define objectives and success criteria before execution.
 - Break complex tasks into manageable phases with clear deliverables.
 - Use a 3–5 step plan to sequence activities, enabling progress tracking and easier debugging.
- Data controls and governance:
 - Data minimization: limit the data fed into prompts to what is strictly necessary.
 - Redaction and sanitization: remove or obfuscate sensitive information (PII, secrets) before processing.
 - Access controls: ensure only authorized users and trusted environments interact with AI workflows.
 - Data retention and deletion policies: outline how long prompts and outputs are stored and when they're purged.
- Operational practices:
 - Versioning of prompts and data schemas to maintain traceability.
 - Logging and audit trails for prompts, model versions, and decisions.
 - Clear ownership: assign responsibility for data handling, model behavior, and compliance checks.
- Example structure shown:
 - Context: Brief description of the task and constraints.
 - Task: Specific steps or sub-tasks to accomplish the goal.
 - Data Controls: Rules for data minimization, redaction, access, and retention.
- Outputs and applicability:
 - Aimed at teams implementing large or sensitive AI workflows, such as product teams, data science, or security/compliance functions.
 - Provides guidance you can apply to design prompts, govern data flows, and ensure auditable decisions.
- Best practices highlighted:
 - Start with a formal plan and a data-control checklist before running prompts.
 - Maintain separation between development and production environments.
 - Include privacy and security guardrails in every prompt and workflow design.

Hallucinations

- Hallucinations are generated outputs that are factually incorrect, misleading, or not grounded in the input data. The video emphasizes identifying, mitigating, and designing prompts and workflows to minimize these errors in AI systems.
- Why hallucinations occur:
 - Data gaps: the model fills in missing information with plausible but unverified content.
 - Over-generalization: applying patterns from training data to inappropriate contexts.
 - Ambiguity in prompts: vague requests lead the model to invent details to satisfy the query.
- Detection strategies:
 - Cross-check outputs against reliable sources or internal data.
 - Use structured prompts that require evidence, sources, or verifiable steps.
 - Implement red-teaming prompts to probe for unsupported claims, contradictions, or edge cases.
- Mitigation techniques:
 - Prompt design: request citations, explicit boundaries (what is in scope vs. not), and conservative language.
 - Verification steps: add a secondary validation stage or an external reasoning check.
 - Tool-assisted grounding: fetch data from trusted sources or databases to anchor responses.
- Practical prompt patterns:
 - Request evidence: "Provide a concise answer and include at least two verifiable sources."
 - Limit scope: "Answer only based on the data in [specified dataset] and do not generalize beyond it."
 - Guardrails: "If uncertain, flag the area and suggest how to verify."
- Best practices highlighted:
 - Build outputs with transparency: clearly separate conclusions from inferred reasoning.
 - Encourage cautious language in uncertain areas.
 - Regularly audit prompts and outputs for accuracy and bias, updating guardrails as needed.
- Examples illustrated:
 - A prompt that asks for a summary with explicit source attributions and caveats.

- A workflow where AI generates a claim, then a human reviewer checks for factual grounding before final delivery.
- Outcomes to aim for:
 - More trustworthy AI responses with explicit grounding.
 - Reduced risk of spreading misinformation.
 - Clear processes for validation and correction when errors are detected.